

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Gabriel Castão Graciano

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	4
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	6
4.4	Atividade 4 — Controle de acesso por papel — RBAC	8
4.5	Atividade 5 — Logs e auditoria	10
4.6	Atividade 6 — Upload e perfil de usuário	11
5	Atividades extras (opcional)	13
5.1	Atividade E1 — Documentação Swagger/OpenAPI	13
5.2	Atividade E2 — CI/CD com GitHub Actions	15
5.3	Atividade E3 — Observabilidade — health checks e métricas	17
6	Considerações finais	18
7	Declaração de autoria	19

1 Introdução

A disciplina de Introdução à Computação em Nuvem (ISW055) tem como objetivo preparar o estudante para compreender, projetar e operar arquiteturas distribuídas. No primeiro bimestre de 2026.2, o aprendizado foi essencialmente prático, cobrindo isolamento de ambientes com Docker, orquestração com Docker Compose, autenticação centralizada, controle de acessos, observabilidade e armazenamento de objetos.

O projeto central do período foi o Catálogo de Filmes do ator Tom Hanks, desenvolvido de forma iterativa e modular. As atividades efetivamente documentadas neste relatório começam pela Atividade 2; a Atividade 1, de nivelamento em Flask, não foi entregue. A partir da Atividade 2, o catálogo foi organizado em múltiplos microsserviços conectados em uma rede interna, com consumo da API do TMDb, autenticação via JWT, persistência relacional, registro centralizado de eventos de auditoria e gestão de arquivos com MinIO. Das nove atividades previstas (seis obrigatórias e três extras), sete foram entregues (a Atividade 2 com atraso), uma foi entregue parcialmente (E2, sem deploy automático) e uma não foi realizada (Atividade 1).

Este relatório bimestral (P1) documenta o progresso realizado. Ele está organizado em: metodologia de validação, cronograma com prazos previstos e efetivos, fichas técnicas por atividade (arquitetura, commits, capturas de tela e resolução de problemas) e considerações finais sobre o que foi aprendido.

2 Metodologia

As atividades efetivamente realizadas e descritas neste relatório foram versionadas em um único repositório público no GitHub (`catalogo-tom-hanks`), com commits descritivos. A Atividade 1 não foi realizada. A arquitetura documentada rodou em ambiente Linux/Docker, orquestrada por arquivos `docker-compose.yml` e gerenciada pelo Portainer, com comunicação entre componentes restrita a uma rede interna isolada (`tom_hanks_net`).

As evidências foram coletadas com foco em rastreabilidade e verificação externa. As datas e horários das entregas vieram do histórico de commits (`git log` e interface web do GitHub). Esses commits são, em sua maioria, commits de documentação (docs), que registram o momento em que as evidências foram publicadas no README, e não necessariamente o momento em que o código foi implementado. Por isso a ordem cronológica entre algumas atividades pode parecer invertida: a Atividade 3, por exemplo, consta em 23/08/2026, antes da publicação da documentação da Atividade 2, em 12/09/2026.

As capturas funcionais (interface web, Swagger UI, terminal e painel do Portainer) foram, em sua maioria, refeitas entre 02/10/2026 e 04/10/2026 para este relatório; a página do catálogo da Atividade 2 e o painel do GitHub Actions da Atividade E2 foram capturados em data anterior. Nelas, os horários dos cabeçalhos HTTP e dos logs estão em UTC/GMT, três horas à frente do horário de Brasília. Entre algumas capturas, o banco de dados de testes foi recriado, de modo que identificadores e nomes de usuário podem diferir de uma figura para outra. Senhas, tokens e chaves foram ocultados nas imagens.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	-	não entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência relacional (SQLite na stack; código compatível com MariaDB) e segregação por usuário.	20/08/2026	12/09/2026 20:00	entregue com atraso
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	23/08/2026 14:32	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	29/08/2026 21:39	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	05/09/2026 22:08	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	18/09/2026 19:21	entregue
E1	Documentação Swagger/OpenAPI	Swagger UI com pelo menos 2 serviços documentados e 'Try it out' executado.	sem prazo	11/09/2026 20:06	entregue
E2	CI/CD com GitHub Actions	CI configurada para testar e publicar imagens no GHCR a cada push; deploy da stack manual pelo Portainer.	sem prazo	11/09/2026 21:11	parcial — deploy manual
E3	Observabilidade — health checks e métricas	Endpoints /health e /metrics; indisponibilidade do Redis sinalizada como unhealthy no log_service; o catálogo permanece respondendo. O health check não reinicia automaticamente o container.	sem prazo	12/09/2026 19:35	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
------------------	---

Data planejada	07/08/2026
Data realizada	-
Situação	não entregue
Evidência	Não se aplica (atividade não realizada)
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks

O que faltou e motivo. Não realizei esta atividade porque faltei à aula presencial em que ela foi ensinada. Para manter o cronograma, optei por não refazê-la depois e concentrar os esforços no catálogo em microsserviços, a partir da Atividade 2.

Dificuldades e como foram resolvidas. Não se aplica, já que a atividade não foi feita. O conteúdo sobre rotas e persistência foi retomado e consolidado nas atividades seguintes, com FastAPI e Docker.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDb, persistência relacional (SQLite na stack; código compatível com MariaDB) e segregação por usuário.
Data planejada	20/08/2026
Data realizada	12/09/2026 20:00
Situação	entregue com atraso
Evidência	GitHub — commit ddefcc2, README, catálogo em execução, lista de favoritos e comparação entre duas contas
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/commit/ddefcc2

O que foi feito. Desenvolvi a aplicação do catálogo temático de Tom Hanks. O backend usa FastAPI e consulta a API do The Movie Database (TMDb) para obter títulos, sinopses e pôsteres. Implementei persistência relacional para armazenar favoritos e comentários por usuário. O código aceita MariaDB (driver pymysql) ou a DATABASE_URL configurada; na stack documentada, ela aponta para SQLite. A aplicação foi containerizada com Docker. A Figura 2 demonstra a interface e a listagem apresentada no ambiente consultado; como alguns títulos visíveis não são claramente associáveis à filmografia de Tom Hanks, essa captura, por si só, não comprova um filtro exclusivo pelo ator. A Figura 3 mostra os favoritos de uma sessão autenticada, e a Figura 4 compara duas contas: com o token de cada uma, o usuário comum e o administrador recebem listas distintas de favoritos (5 e 2 títulos), e o administrador não recebe os filmes que apenas o usuário comum favoritou, o que demonstra a segregação por usuário. A documentação e as evidências foram consolidadas em 12/09/2026 às 20:00 (commit ddefcc2).

```
commit ddefcc274af852bec724a4bae73c9c58e3ac2117
Author: Gabriel Castão Graciano <gbgraciano@hotmail.com>
Date: Sat Sep 12 20:00:30 2026 -0300

docs: documentacao completa da atividade catalogo de filmes tom hanks

diff --git a/README.md b/README.md
index fd86f0b..1cf7c9c 100644
--- a/README.md
+++ b/README.md
@@ -298,9 +298,10 @@ A aplicação monolítica original foi desacoplada em uma **Arquitetura de Micro
### 3. Bloqueio de Segurança com Token Inválido/Expirado
![Bloqueio Token Inválido](https://raw.githubusercontent.com/Gabriel33-fis/catalogo-tom-hanks/main/prints/print_3_token_invalid.png)
```

Figura 1: Atividade 2 — evidência da entrega (commit ddefcc2 com data e hora visíveis)

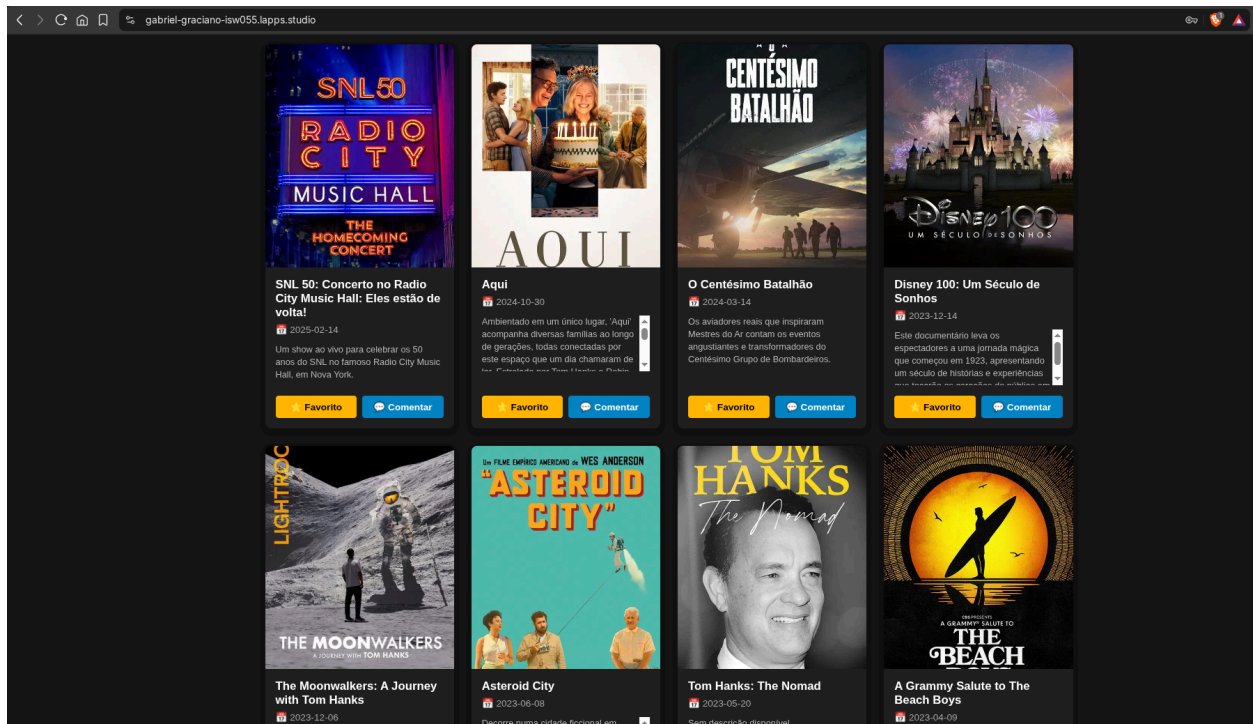


Figura 2: Atividade 2 — resultado (interface e listagem do catálogo em execução; a captura não comprova, isoladamente, filtro exclusivo pela filmografia de Tom Hanks)

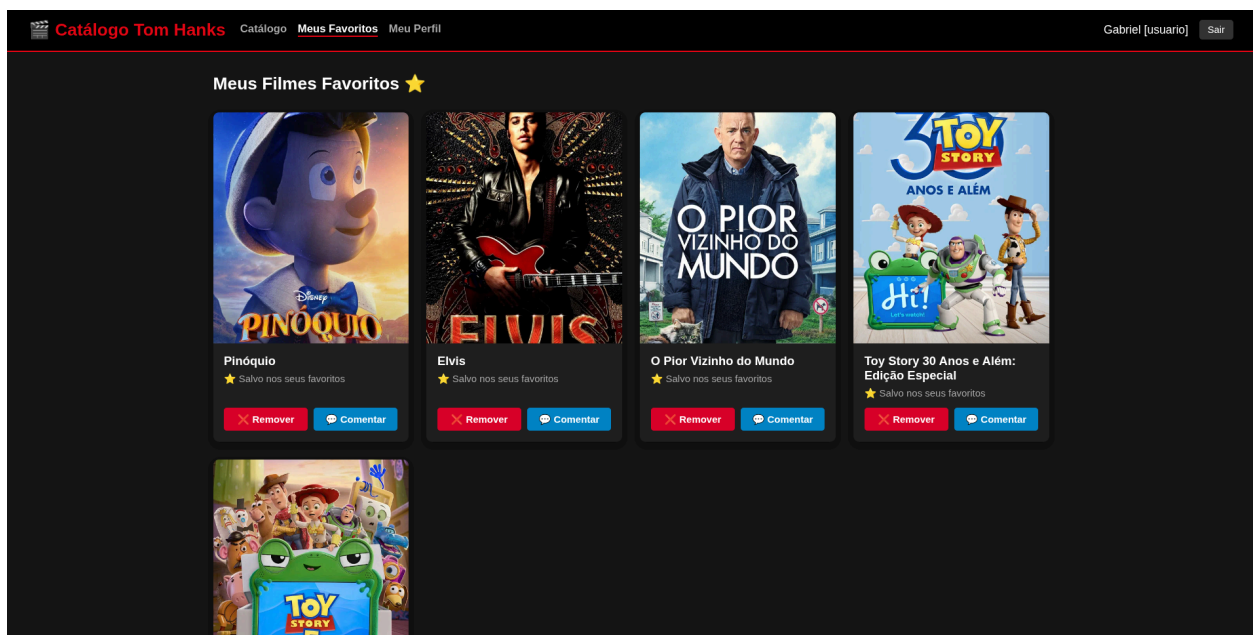


Figura 3: Atividade 2 — resultado complementar (página Meus Favoritos da sessão autenticada)

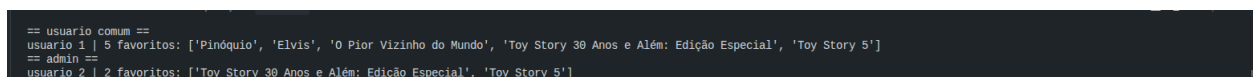


Figura 4: Atividade 2 — resultado (segregação por usuário: o GET /api/perfil/me com o token de cada conta retorna 5 favoritos para o usuário 1, papel usuario, e apenas 2 para o usuário 2, papel admin, sem os demais filmes do primeiro)

Dificuldades e como foram resolvidas. A principal dificuldade foi tratar erros na comunicação síncrona com a API do TMDb e organizar a segregação das preferências por usuário. Resolvi isso com um módulo desacoplado para as consultas à API e passando as credenciais por variáveis de ambiente no container, e não no código-fonte.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	23/08/2026 14:32
Situação	entregue
Evidência	GitHub — commit 26fde5e e fluxo de recuperação de senha (formulário, e-mail no Mailtrap e redefinição)
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/commit/26fde5e

O que foi feito. Isolei a gestão de identidade em um microsserviço dedicado (auth_service), responsável pelo registro de usuários, validação de credenciais com hashing seguro de senhas, emissão e validação de tokens JWT (Bearer) e recuperação de senha integrada ao Mailtrap via SMTP, em ambiente de testes. A comunicação entre os serviços ficou restrita à rede interna do Docker (tom_hanks_net), deixando apenas a porta da aplicação web acessível externamente. O serviço e as evidências foram consolidados em 23/08/2026 às 14:32 (commit 26fde5e). O login e a emissão do token JWT estão evidenciados nas figuras da Atividade E1 (resposta 200 com access_token e papel do usuário), e o fluxo de recuperação de senha, nas figuras abaixo. Essas capturas demonstram os fluxos funcionais; não comprovam, por si sós, o isolamento da rede Docker, as portas expostas ou o hashing das senhas, que devem ser verificados no docker-compose.yml e no código-fonte.

```
commit 26fde5e285d1ecf4c1b0e1e698499d570479c922
Author: Gabriel Castão Graciano <gbgraciano@hotmail.com>
Date: Sun Aug 23 14:32:27 2026 -0300

    docs: adiciona imagens de evidencia da recuperacao de senha

diff --git a/README.md b/README.md
index fb2f8d4..0eee4eb 100644
--- a/README.md
+++ b/README.md
@@ -14,13 +14,26 @@
...
+## 📸 Evidências de Funcionamento (Atividade 3)
```

Figura 5: Atividade 3 — evidência da entrega (commit 26fde5e com data e hora visíveis)

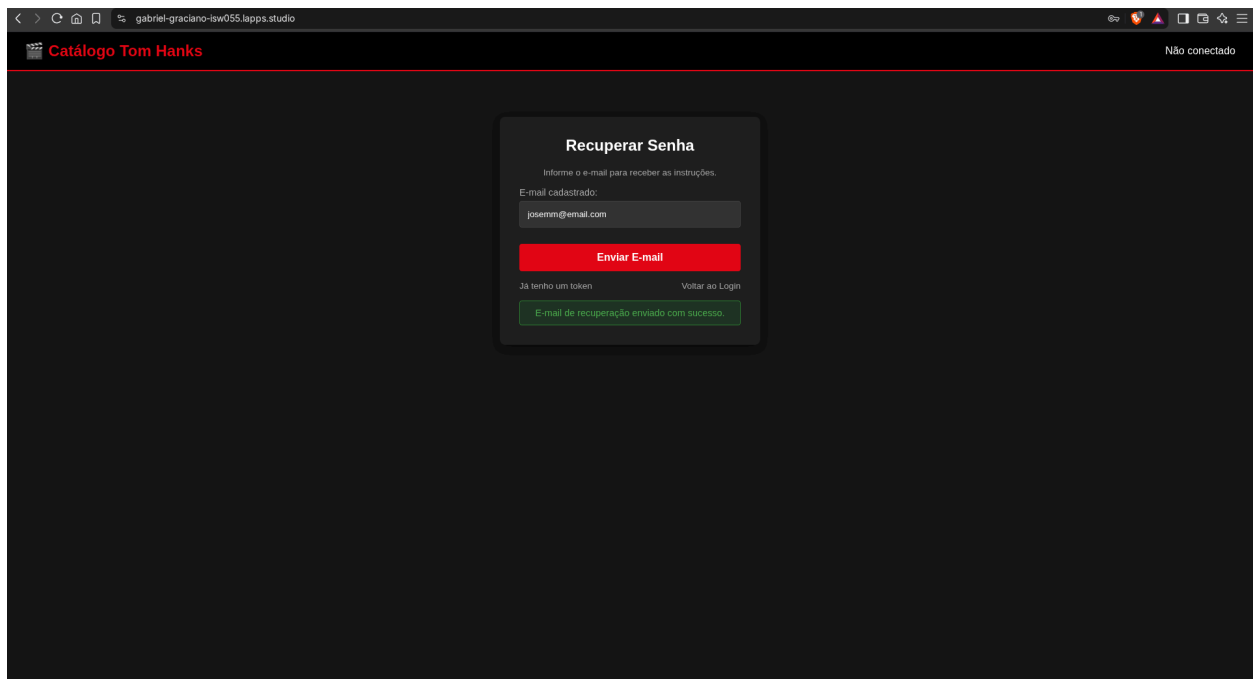


Figura 6: Atividade 3 — resultado (recuperação de senha: formulário com a confirmação de envio do e-mail)

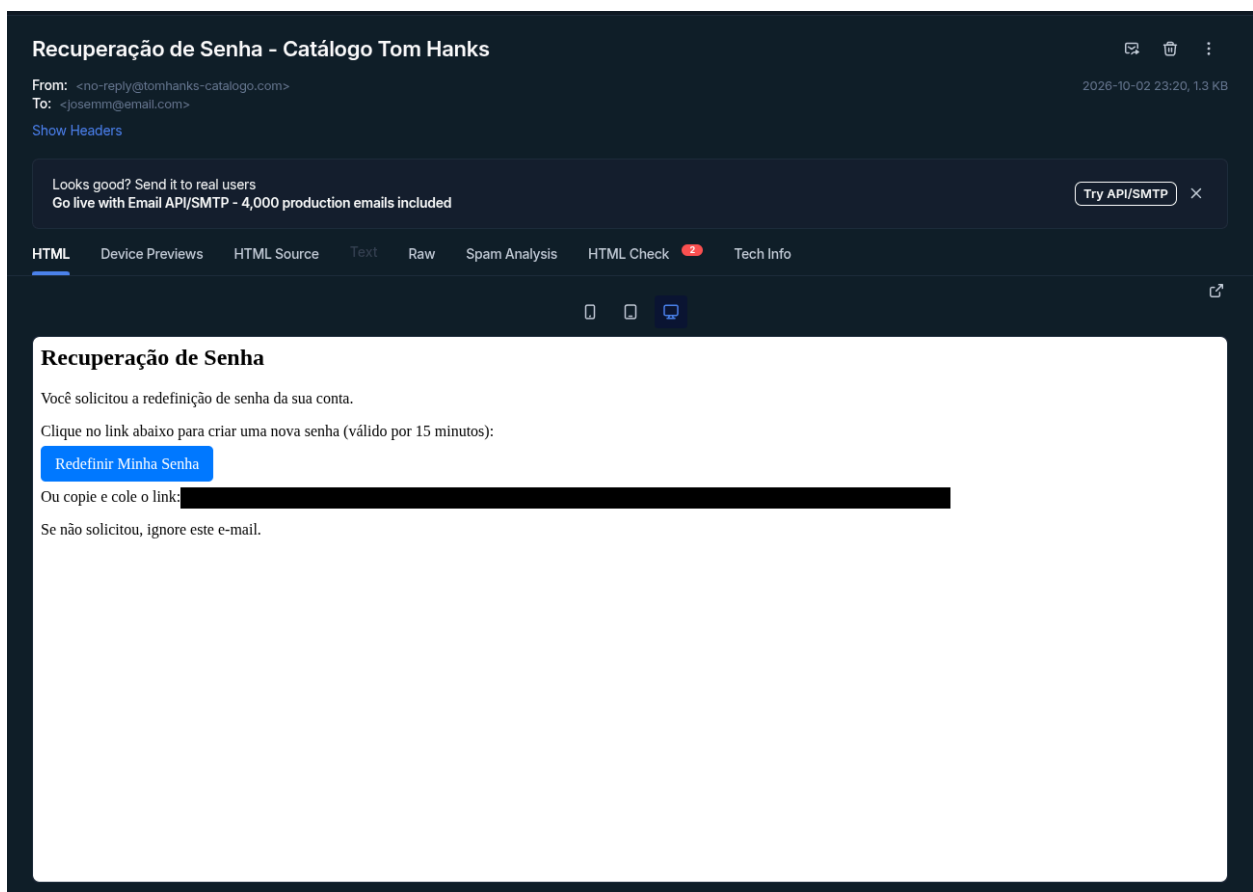


Figura 7: Atividade 3 — resultado (recuperação de senha: e-mail recebido na caixa de testes do Mailtrap, com o link de redefinição; token ocultado; horário em UTC)

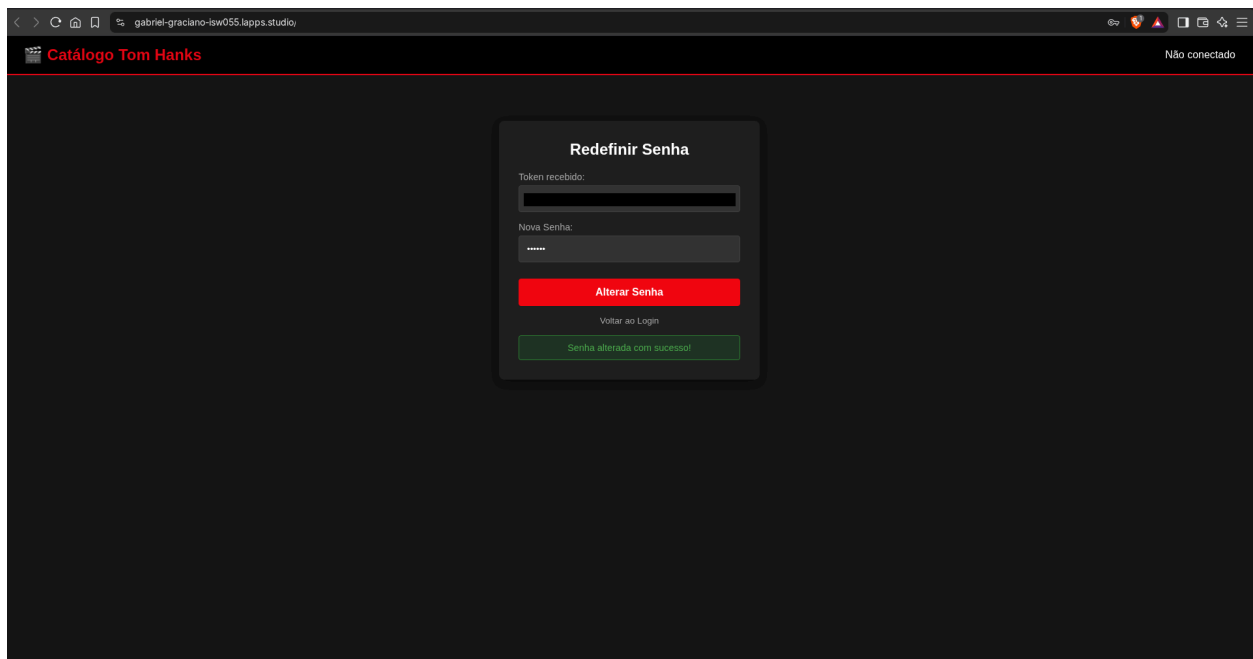


Figura 8: Atividade 3 — resultado (recuperação de senha: formulário de redefinição aberto pelo link, com a mensagem de sucesso; token ocultado)

Dificuldades e como foram resolvidas. O principal desafio foi garantir que os outros serviços validassem a assinatura dos tokens JWT sem depender do banco de dados do `auth_service`. Resolvi adotando uma chave secreta compartilhada por variáveis de ambiente no `docker-compose.yml`, e desacoplando o envio de e-mails para tratar falhas de rede na comunicação com o servidor SMTP. Na validação final, o envio do e-mail de recuperação falhava no login SMTP porque as credenciais do Mailtrap não chegavam ao container (as variáveis `MAILTRAP_*` não estavam definidas no serviço). Identifiquei a causa pelo log do container e corriji definindo essas variáveis no serviço de autenticação.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo <code>role</code> passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	29/08/2026 21:39
Situação	entregue
Evidência	GitHub — commit <code>a75b0c4</code> ; interface por papel, GET <code>/api/admin/logs</code> e DELETE de comentário (403 usuário/200 admin)
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/commit/a75b0c4

O que foi feito. Implementei o controle de acesso baseado em papéis (RBAC), associando permissões ao papel do usuário autenticado (`admin` ou `usuario`). As Figuras 10 e 11 mostram a interface: o botão Excluir aparece conforme o papel e a autoria do comentário. A Figura 12 mostra o backend aplicando a regra na consulta GET `/api/admin/logs`, com 403 Forbidden para o usuário comum e 200 OK para o administrador. A Figura 13 mostra a exclusão de um comentário de teste de outro usuário feita diretamente no endpoint DELETE `/api/comentarios/{comentario_id}`: o usuário comum recebe 403 Forbidden e o administrador consegue excluir o comentário (200 OK). O código foi consolidado e as evidências publicadas em 29/08/2026 às 21:39 (commit `a75b0c4`).

```
commit a75b0c4d515e1d09054955e14f33b19c142aab51
Author: Gabriel Castão Graciano <gbgraciano@hotmail.com>
Date: Sat Aug 29 21:39:02 2026 -0300

docs: garante envio de todas as imagens da pasta prints

diff --git a/README.md b/README.md
index ee22598..c708ed4 100644
--- a/README.md
+++ b/README.md
@@ -67,6 +67,17 @@
@@ A aplicação monolítica original foi desacoplada em uma **Arquitetura de Micro
```

Figura 9: Atividade 4 — evidência da entrega (commit a75b0c4 com data e hora visíveis)

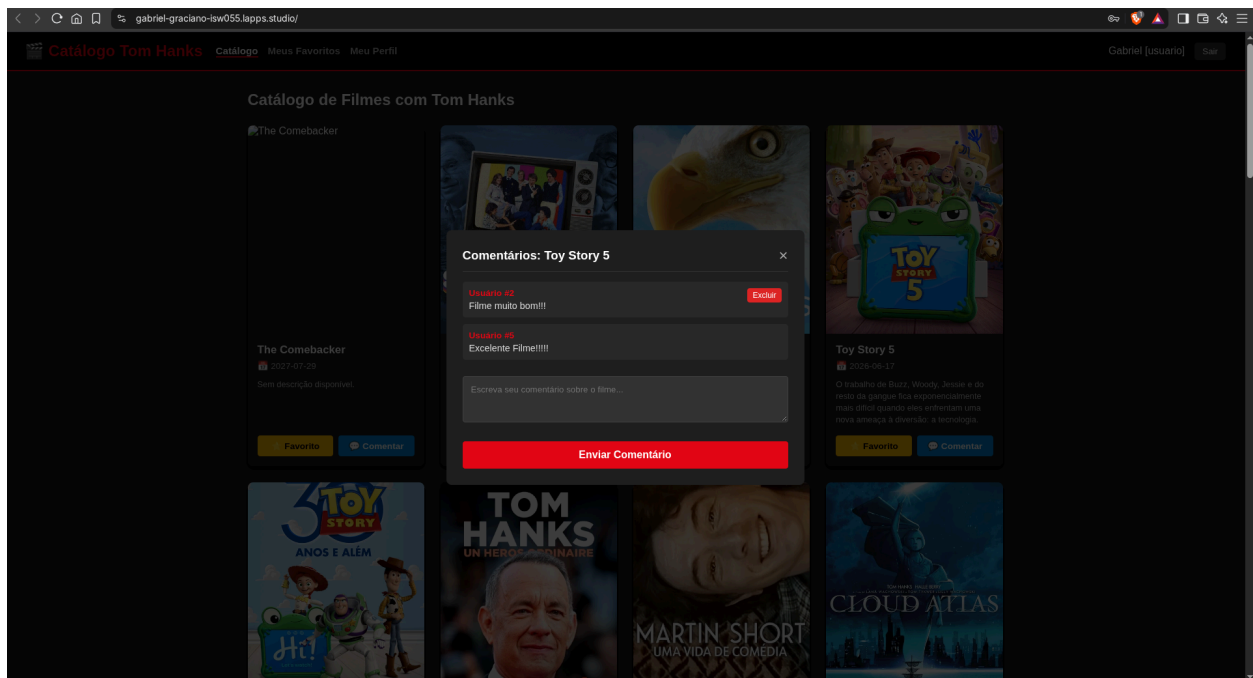


Figura 10: Atividade 4 — resultado (interface web com o papel usuário: o botão Excluir aparece apenas no comentário de autoria do próprio usuário)

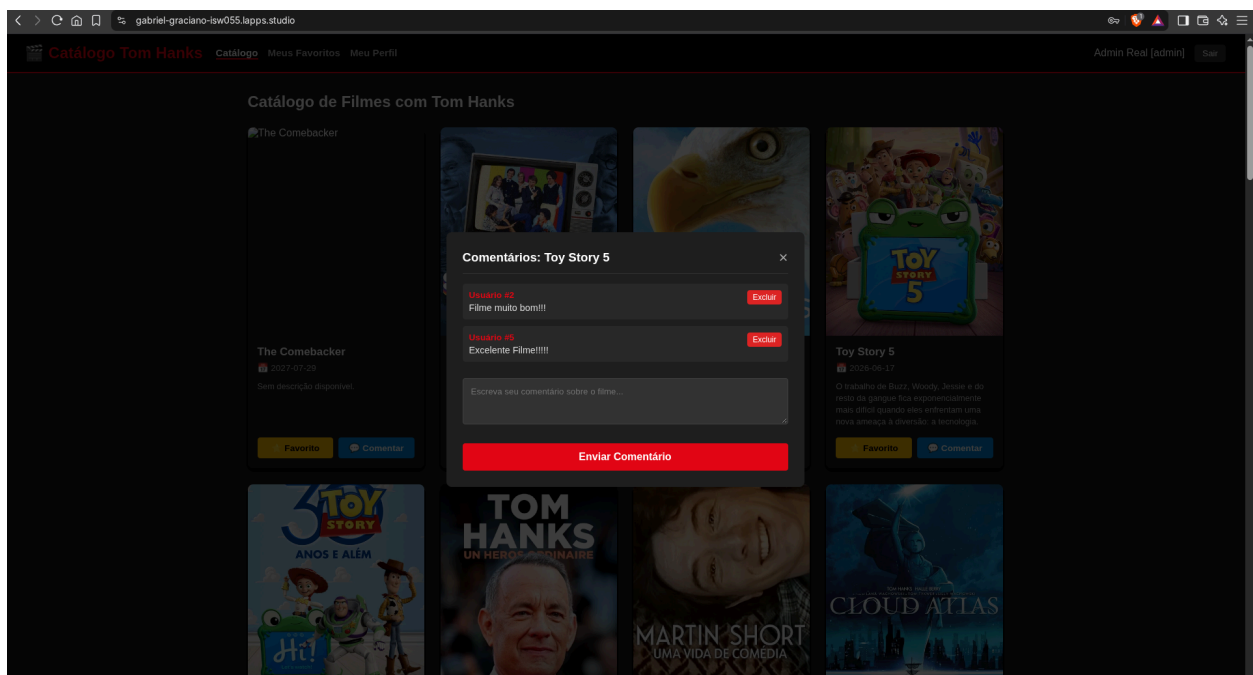


Figura 11: Atividade 4 — resultado (interface web com o papel admin: o botão Excluir aparece em todos os comentários)

```

Problemas Saida Console de Depuração Terminal Portas
== USUÁRIO COMUM ==
HTTP 403
Acesso negado (403 Forbidden): apenas administradores podem acessar os logs de auditoria.
== ADMIN ==
HTTP 200
total de eventos: 50
2026-10-02T23:58:14 | usuario 2 | tentativa_negada_403
2026-10-02T23:49:38 | usuario 4 | comentar
2026-10-02T23:49:29 | usuario 4 | login
2026-10-02T23:46:25 | usuario 2 | tentativa_negada_403
2026-10-02T23:45:59 | usuario 4 | login
2026-10-02T23:45:20 | usuario 2 | login

```

Figura 12: Atividade 4 — resultado (mesma requisição GET /api/admin/logs: 403 Forbidden para o papel usuario e 200 OK para o papel admin; horários em UTC)

```

Problemas Saida Console de Depuração Terminal Portas
== usuario comum tenta excluir o comentario de OUTRO usuario (id 1) ==
{"detail":"Acesso negado (403 Forbidden): apenas administradores podem apagar comentários de outros usuários."}
HTTP 403
== admin exclui o mesmo comentario ==
{"message":"Comentário removido com sucesso"}
HTTP 200

```

Figura 13: Atividade 4 — resultado (DELETE /api/comentarios/{comentario_id} em um comentário de teste de outro usuário: 403 Forbidden para o papel usuario e 200 OK para o papel admin)

Dificuldades e como foram resolvidas. O principal desafio foi interceptar requisições não autorizadas sem duplicar a lógica de verificação em cada endpoint. Resolvi usando **dependencies** reutilizáveis no FastAPI (Depends), centralizando a leitura e a validação do token JWT em um módulo (auth_guard) e o tratamento das exceções HTTPException(status_code=403) nas rotas restritas.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	05/09/2026 22:08
Situação	entregue
Evidência	GitHub — commit f1cb693 e consulta dos logs de auditoria pelo administrador
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/commit/f1cb693

O que foi feito. Desenvolvi um novo microserviço de auditoria (log_service), apoiado em uma instância containerizada de Redis, que armazena os eventos em uma estrutura do tipo Stream. O serviço registra eventos críticos, como logins, logouts, favoritos, publicação de comentários e tentativas de acesso negadas com código 403 Forbidden. Também disponibilizei um endpoint protegido, restrito a administradores (GET /api/admin/logs), que permite consultar os eventos mais recentes, com limite configurável de quantidade. A entrega e as evidências foram registradas em 05/09/2026 às 22:08 (commit f1cb693).

```

commit f1cb6934b45783303835f9bfeb48f8ce17c95288
Author: Gabriel Castão Graciano <gbgraciano@hotmail.com>
Date: Sat Sep 5 22:08:27 2026 -0300

docs: adiciona prints da atividade 5 no readme

diff --git a/README.md b/README.md
index 470903f..f57e5d3 100644
--- a/README.md
+++ b/README.md
@@ -14,8 +14,6 @@
---

```

Figura 14: Atividade 5 — evidência da entrega (commit f1cb693 com data e hora visíveis)

```

4) "logout"
5) "timestamp"
6) "2026-10-04T12:28:43.033234+00:00"
7) "ip_origem"
8)
9) "detalhes"
10) "Logout efetuado com sucesso"
3) 1) "1791109510560-0"
2) 1) "usuario_id"
2) "i"
3) "acao"
4) "login"
5) "timestamp"
6) "2026-10-04T10:25:10.559973+00:00"
7) "ip_origem"
8)
9) "detalhes"
10) "Login realizado pelo usu\x3\xairio 'Gabriel' [usuario]"
4) 1)
2) 1) "usuario_id"
2) "i"
3) "acao"
4) "favoritar_filme"
5) "timestamp"
6) "2026-10-04T10:14:35.773877+00:00"
7) "ip_origem"
8)
9) "detalhes"
10) "Favoritou o filme 'Pin\x3\xb3quio' (TMDb ID: 532639)"
/data #

```

Figura 15: Atividade 5 — resultado (Redis Stream audit_logs consultado no container gabriel_redis com XREVRANGE: eventos recentes de login, logout e favorito, com ação, horário e detalhes; horários em UTC)

Dificuldades e como foram resolvidas. O principal desafio foi registrar os eventos vindos de múltiplos serviços sem afetar a latência das requisições principais do usuário. Resolvi usando operações eficientes no Redis, com timeout curto nas chamadas ao serviço de logs, e ajustando as dependências de rede no Docker Compose, de forma que falhas momentâneas na auditoria não interrompessem o catálogo (comportamento observado, de forma parcial, na Atividade E3, em que o catálogo seguiu respondendo com o Redis parado).

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	18/09/2026 19:21
Situação	entregue
Evidência	GitHub — commit f8d5442; perfil, proxy/MinIO, referência da foto no banco e edição própria/alheia com respostas 200/403
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/commit/f8d5442

O que foi feito. Implementei o perfil de usuário com upload de avatar para o MinIO e acesso à imagem por um endpoint proxy do backend. As Figuras 18, 19 e 21 mostram, respectivamente, a resposta do endpoint com o campo foto_url e a entrega da imagem, a presença de um objeto no bucket perfil-fotos e os resultados de autorização para edição: 200 OK no próprio perfil e 403 Forbidden ao tentar editar o perfil de outro usuário. A Figura 20 mostra a consulta direta ao banco do catálogo: a coluna da foto guarda apenas uma referência em texto (perfil_1.png), enquanto o arquivo de 15274 bytes está no bucket do MinIO. A Figura 22 mostra a validação do upload: um arquivo .txt e uma imagem de 3 MB são recusados pela API.. A entrega e as evidências foram concluídas em 18/09/2026 às 19:21 (commit f8d5442).

```

commit f8d54424b9a9ff37dc8491a12a2b729a0eb71389 (HEAD -> main, origin/main)
Author: Gabriel Castão Graciano <gbgraciano@hotmail.com>
Date: Fri Sep 18 19:21:58 2026 -0300

    docs: adiciona evidencias da atividade 6 e documentacao no README

diff --git a/README.md b/README.md
index 1cf7c9c..d564dcf 100644
--- a/README.md
+++ b/README.md
@@ -13,6 +13,25 @@
 * **Perfil do Professor:** [github.com/siriani](https://github.com/siriani)

```

Figura 16: Atividade 6 — evidência da entrega (commit f8d5442 com data e hora visíveis)

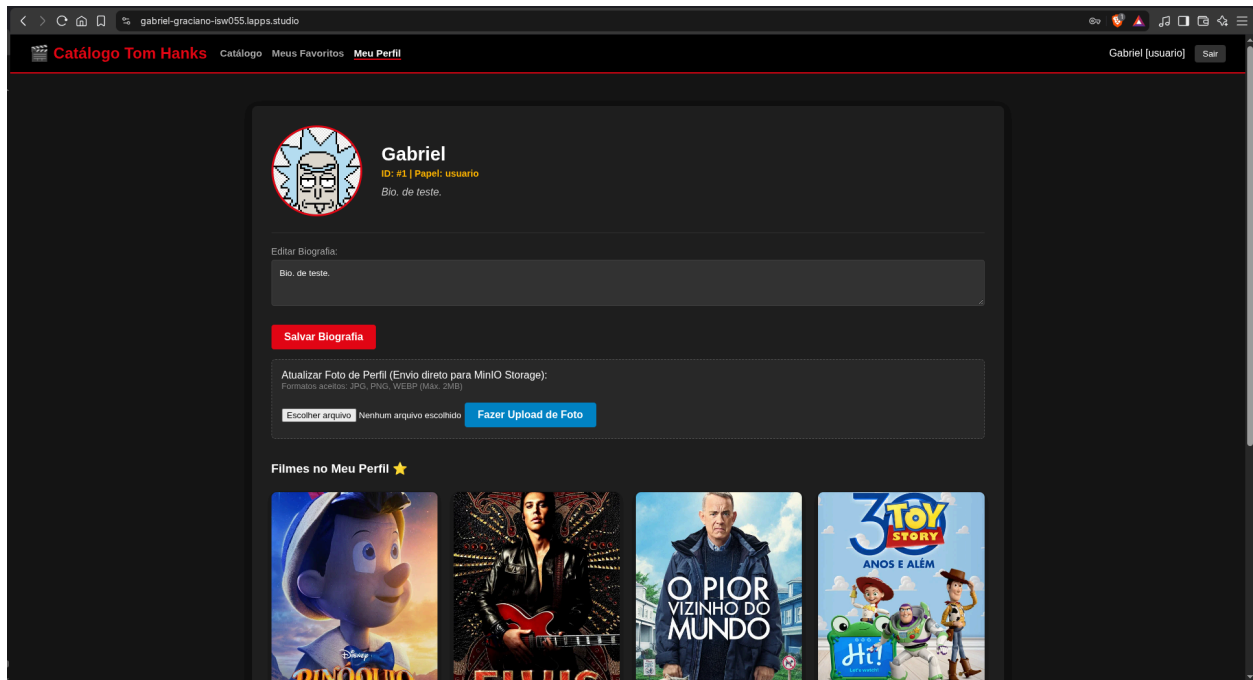


Figura 17: Atividade 6 — resultado (página de perfil do usuário autenticado, com avatar, edição de biografia, envio de foto para o MinIO e filmes favoritos)



Figura 18: Atividade 6 — resultado (o GET /api/perfil/me retorna o campo foto_url, e o endpoint proxy do backend entrega a imagem guardada no MinIO: 200, image/png, 15274 bytes, requisição com o token do usuário)

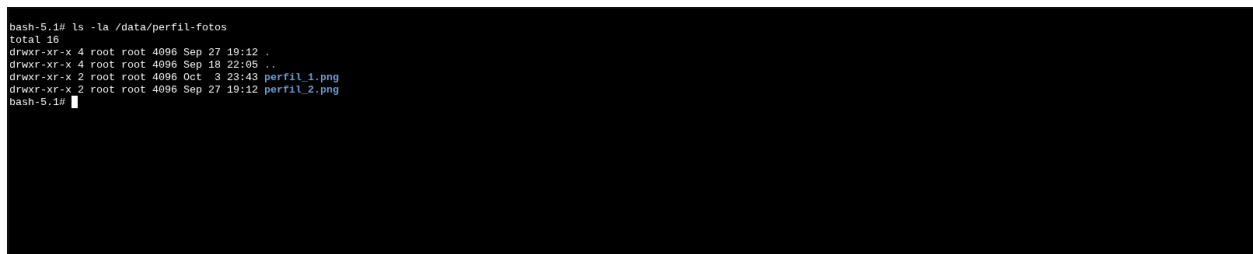


Figura 19: Atividade 6 — resultado (console do container gabriel_minio: o bucket perfil-fotos contém um objeto por usuário, nomeado pelo identificador, como perfil_1.png; horários em UTC)

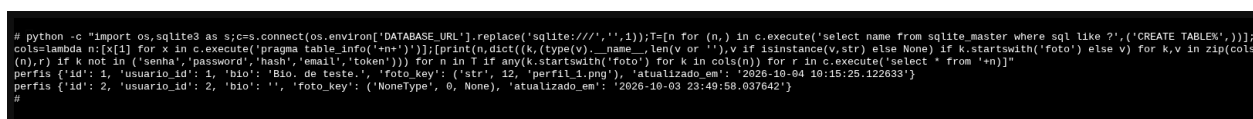


Figura 20: Atividade 6 — resultado (consulta ao banco do catálogo, no container gabriel_catalogo_service: a coluna da foto guarda apenas a referência em texto, e não o arquivo, que está no bucket do MinIO; colunas sensíveis omitidas)

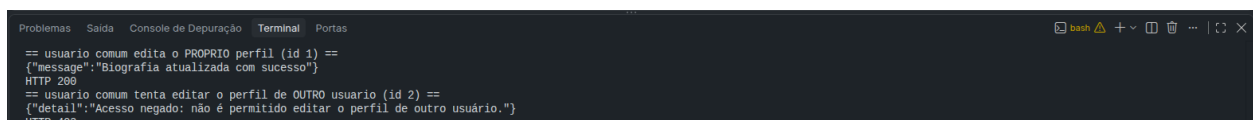


Figura 21: Atividade 6 — resultado (controle de acesso no perfil: o usuário comum edita o próprio perfil, 200 OK, e recebe 403 Forbidden ao tentar editar o perfil de outro usuário)

```

Problemas  Saída  Console de Depuração  Terminal  Portas
== upload de arquivo .txt (tipo inválido) ==
{"detail": "Tipo de ficheiro inválido. Apenas imagens JPEG, PNG ou WEBP são permitidas."}
HTTP 400
== upload de imagem com 3 MB (acima do limite de 2 MB) ==
{"detail": "Ficheiro excede o tamanho máximo permitido de 2 MB."}
HTTP 400

```

Figura 22: Atividade 6 — resultado (validação do upload: um arquivo .txt e uma imagem de 3 MB, acima do limite de 2 MB, são recusados pela API)

Dificuldades e como foram resolvidas. O principal desafio foi isolar a rede do MinIO sem expor a API de objetos diretamente à internet, mantendo o streaming de dados seguro pela própria aplicação FastAPI. Resolvi integrando o MinIO à rede interna `tom_hanks_net` e criando um endpoint proxy no backend que recupera os bytes sob demanda, após validar o token do usuário. Essa descrição corresponde à configuração e ao comportamento implementados; as capturas atuais demonstram a resposta do proxy, mas não exibem a configuração de rede ou de portas.

5 Atividades extras (opcional)

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Swagger UI com pelo menos 2 serviços documentados e 'Try it out' executado.
Data planejada	sem prazo
Data realizada	11/09/2026 20:06
Situação	entregue
Evidência	GitHub — commit dc38e5a, Swagger UI com autenticação Bearer (Try it out em login e perfil) e OpenAPI dos serviços internos
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/commit/dc38e5a

O que foi feito. Enriqueci os contratos HTTP com anotações e modelos Pydantic no FastAPI, de modo que o Swagger UI documenta parâmetros de entrada, corpos de requisição e respostas, incluindo a resposta 401 Unauthorized para token ausente, inválido ou expirado. A documentação única do catálogo, que é o ponto de entrada da aplicação, reúne as rotas `/api/...` de autenticação, catálogo, perfil e armazenamento, comentários e auditoria. Além disso, o `auth_service` (`/register`, `/login`, `/forgot-password` e `/reset-password`) e o `log_service` (`/logs`) publicam o próprio OpenAPI na rede interna, e todos os serviços expõem `/health` e `/metrics`. A interface foi disponibilizada e testada interativamente com o “Try it out”. A documentação e os esquemas foram enviados em 11/09/2026 às 20:06 (commit dc38e5a); a mensagem desse commit cita a Atividade 6 por engano de numeração, e seu conteúdo é a documentação OpenAPI desta atividade. Em 02/10/2026, ao refazer as evidências, acrescentei o esquema de segurança Bearer (HTTPBearer, commit 6805032), que habilita o botão Authorize no Swagger UI e permite testar rotas protegidas com o token JWT.

```

commit dc38e5a8be5cf93254bf9c65d753f581eef1da6
Author: Gabriel Castão Graciano <gbgraciano@hotmail.com>
Date: Fri Sep 11 20:06:21 2026 -0300

    docs: conclui especificacoes openapi e documentacao da atividade 6

diff --git a/README.md b/README.md
index cbf4557..d70e63b 100644
--- a/README.md
+++ b/README.md
@@ -14,29 +14,51 @@

```

Figura 23: Atividade E1 — evidência da entrega (commit dc38e5a com data e hora visíveis)

```

Problemas Saída Console de Depuração Terminal Portas
commit 6865032cae0d5a4c0bad3dd93a4acc093149f7a3 (HEAD -> main, origin/main)
Author: Gabriel Castão Graciano <gbrgraciano@hotmail.com>
AuthorDate: Fri Oct 2 21:04:36 2026 -0300
Commit: Gabriel Castão Graciano <gbrgraciano@hotmail.com>
CommitDate: Fri Oct 2 21:04:36 2026 -0300

    feat: adiciona esquema Bearer ao Swagger (botão Authorize)

diff --git a/catalogo_service/auth_guard.py b/catalogo_service/auth_guard.py
index 34c327d..de51c33 100644
--- a/catalogo_service/auth_guard.py
+++ b/catalogo_service/auth_guard.py
@@ -2,3 +2,4 @@ import os
import jwt
+from fastapi import Header, HTTPException, status
+from fastapi import Depends, HTTPException, status
+from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

@@ -7,4 +8,7 @@ JWT_ALGORITHM = "HS256"

-def obter_usuario_atual(authorization: str = Header(None)):
-    if not authorization or not authorization.startswith("Bearer "):
+auto_error=False mantém as suas mensagens de erro 401 em vez das padrão do FastAPI
+bearer_scheme = HTTPBearer(auto_error=False, description="Cole apenas o access.token retornado pelo login")
+
+def obter_usuario_atual(credenciais: HTTPAuthorizationCredentials = Depends(bearer_scheme)):
+    if not credenciais or not credenciais.credentials:
+        raise HTTPException(
@@ -13,3 +17,3 @@ def obter_usuario_atual(authorization: str = Header(None)):
    token = authorization.split(" ")[1]
+    token = credenciais.credentials
    try:

```

Figura 24: Atividade E1 — evidência do commit que acrescentou o esquema Bearer ao Swagger UI (commit 6805032, de 02/10/2026 às 21:04, com a alteração em auth_guard.py)

Responses

Curl

```
curl -X 'POST' \
  'https://gabriel-graciano-isw055.lapps.studio/api/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "gabriel@graciano.com",
    "senha": "12345678"
  }'
```

Request URL

https://gabriel-graciano-isw055.lapps.studio/api/auth/login

Server response

Code	Details
200	Response body <pre>{ "access token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1b297Y1IEZm9udWVudD5TerZ2BVBXVI06V49e3itwph2Al0bdYi486NIIiShjwQrsRfJf2D6lsUM0%2BxI23NkTTWxx9EUn69520Kjdn7B7ZKqJovAjvR159Eyn7CvW4AL1T6%2BhjHk%2BkFr6jfp5l9ziw83CdpuTtDPXekN2jSrl0AEEU3P"} "</pre> Response headers <pre>alt-svc: h3=":443"; ma=65400 cf-cache-status: DYNAMIC cf-ray: a670431ed4079-GUO content-encoding: zstd content-type: application/json date: Sat, 03 Oct 2026 00:28:49 GMT nel: {"report to": "cf-nel", "success_fraction": 0.0, "max_age": 604800} priority: u=1; report-to: {"group": "cf-nel", "max_age": 604800, "endpoints": [{"url": "https://a-nel.cloudflare.com/report/v4?s=2077YIEZf9uwnvud5TerZ2BVBXVI06V49e3itwph2Al0bdYi486NIIiShjwQrsRfJf2D6lsUM0%2BxI23NkTTWxx9EUn69520Kjdn7B7ZKqJovAjvR159Eyn7CvW4AL1T6%2BhjHk%2BkFr6jfp5l9ziw83CdpuTtDPXekN2jSrl0AEEU3P"}]} server: cloudflare server-timing: cfExtPri</pre>

Responses

Code	Description	Links
200	Login realizado com sucesso. Retorna access_token JWT e papel do usuário.	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

"string"

Figura 25: Atividade E1 — resultado (Swagger UI: “Try it out” em POST /api/auth/login, com resposta 200 e papel do usuário; credenciais e token ocultados; data do cabeçalho em GMT)

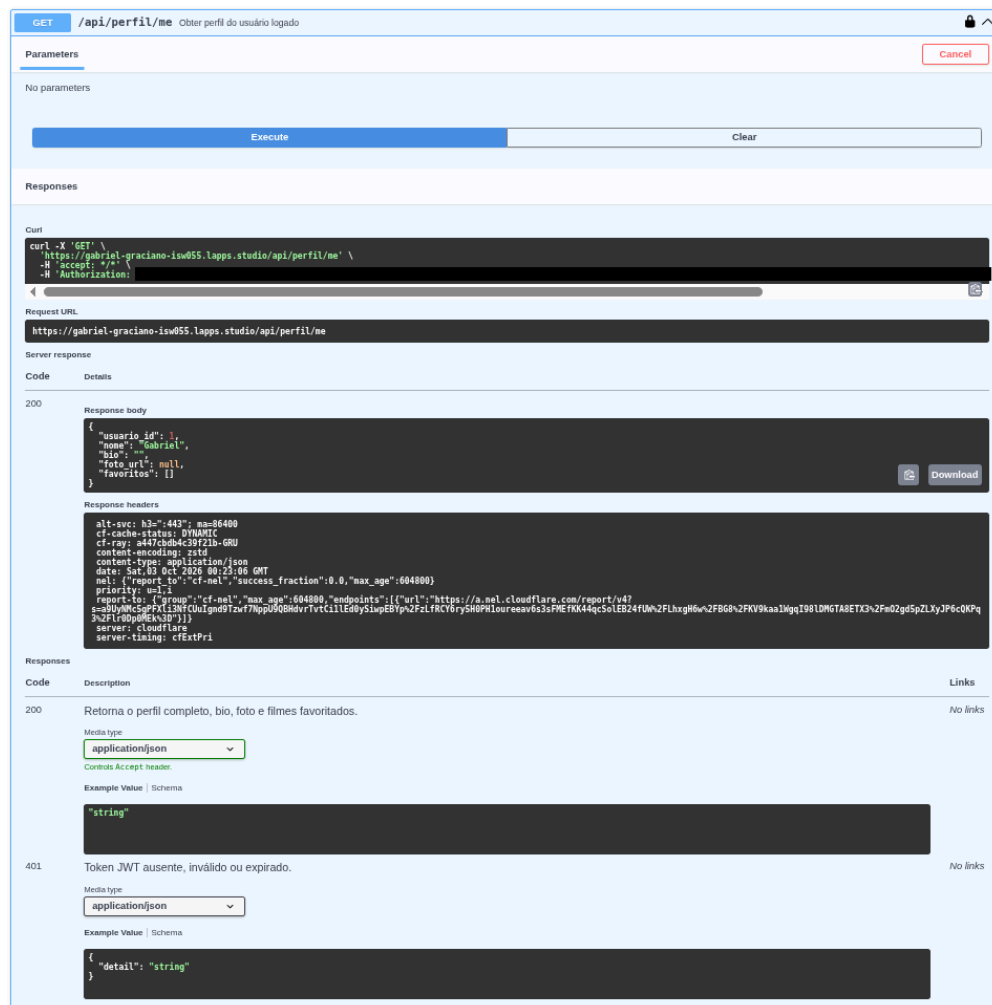


Figura 26: Atividade E1 — resultado (Swagger UI com autorização Bearer: “Try it out” em GET /api/perfil/me, com resposta 200; token ocultado; data do cabeçalho em GMT)

```
root@0a999a866cc0:/app# python -c "import json,urllib.request as u;[print(s,sorted(json.load(u.urlopen('http://%s:%d/openapi.json'%(s,p)))['paths'])) for s,p in (('auth_service',5000),('log_service',6000))]"
auth_service ['/forgot-password', '/health', '/login', '/metrics', '/register', '/reset-password']
log_service ['/health', '/logs', '/metrics']
root@0a999a866cc0:/app#
```

Figura 27: Atividade E1 — resultado (OpenAPI publicado por cada serviço interno, consultado a partir do container do catálogo: o auth_service documenta /register, /login, /forgot-password e /reset-password, e o log_service documenta /logs; ambos com /health e /metrics)

Dificuldades e como foram resolvidas. A principal dificuldade foi autenticar as chamadas protegidas na interface interativa: sem declarar o esquema de segurança, o Swagger UI não oferecia o botão Authorize e não enviava o token nas requisições. Resolvi declarando o esquema Bearer com HTTPBearer na dependência que lê o token, mantendo as mensagens de erro 401 já existentes, e padronizando os modelos de payload de erro com o módulo Pydantic.

5.2 Atividade E2 — CI/CD com GitHub Actions

Descrição	CI configurada para testar e publicar imagens no GHCR a cada push; deploy da stack manual pelo Portainer.
Data planejada	sem prazo
Data realizada	11/09/2026 21:11
Situação	parcial — deploy manual

Evidência	GitHub — commit 97dcb78, workflow e uma execução do Actions concluída com sucesso; atualização da stack manual pelo Portainer.
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/commit/97dcb78

O que foi feito. Configurei no GitHub Actions um workflow de integração contínua para ser disparado em pushes. O pipeline instala as dependências e executa os testes automatizados do `catalogo_service` com `pytest`; quando aprovados, constrói as imagens Docker e publica a imagem no GitHub Container Registry (GHCR), com uma tag baseada no SHA curto do commit. A Figura 29 mostra uma execução concluída com sucesso; isoladamente, ela não comprova o histórico de execuções em cada push. A publicação da imagem é automatizada, mas a atualização dos containers e da stack continua manual, pelo Portainer. Assim, como o deploy não é automático, o requisito de CI/CD permanece parcialmente atendido. O workflow e as evidências foram documentados em 11/09/2026 às 21:11 (commit 97dcb78).

```
commit 97dcb7893e35eb7ade171025bc06ba90f51e6ab6
Author: Gabriel Castão Graciano <gbgraciano@hotmail.com>
Date: Fri Sep 11 21:11:07 2026 -0300

docs: finaliza documentacao e evidencias da atividade extra de ci/cd

diff --git a/README.md b/README.md
index fafa522..1108967 100644
--- a/README.md
+++ b/README.md
@@ -14,10 +14,6 @@
---
```

Figura 28: Atividade E2 — evidência da entrega (commit 97dcb78 com data e hora visíveis)

CI/CD Pipeline - Catalogo Tom Hanks

docs: finaliza documentacao e evidencias da atividade extra de ci/cd #10

Summary

All jobs

- CI - Testes Automatizados
- CD - Build e Publicação no GHCR

Run details

- Usage
- Workflow file

Triggered via push 2 weeks ago

Gabriel33-fis pushed → 97dcb78 main

Status: Success

Total duration: 56s

Artifacts: -

deploy.yml on: push

CI - Testes Automatizados 14s → CD - Build e Publicação no... 35s

Annotations

2 warnings

- CI - Testes Automatizados: Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see: <https://github.blog/ch...> Show more
- CD - Build e Publicação no GHCR: Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, docker/build-push-action@v5, docker/login-action@v3, docker/metadata-acti... Show more

Figura 29: Atividade E2 — resultado (uma execução do Actions com jobs concluídos; comprova o sucesso desta execução, não, isoladamente, a execução em cada push nem o deploy da stack)

Dificuldades e como foram resolvidas. Os testes falhavam no ambiente efêmero do GitHub Actions, que não tem servidor de banco de dados nem as dependências do projeto. O código usa o driver `pymysql` para o MariaDB, e o job de testes não declara serviço de banco. Resolvi instalando no workflow as dependências do `catalogo_service` e, explicitamente, `pytest`, `httpx` e `pymysql`, e mantendo os testes independentes de um servidor de banco de dados.

5.3 Atividade E3 — Observabilidade — health checks e métricas

Descrição	Endpoints /health e /metrics; indisponibilidade do Redis sinalizada como unhealthy no log_service; o catálogo permanece respondendo. O health check não reinicia automaticamente o container.
Data planejada	sem prazo
Data realizada	12/09/2026 19:35
Situação	entregue
Evidência	GitHub — commit a57e484, endpoints /health e /metrics e painel do Portainer com o Redis parado
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/commit/a57e484

O que foi feito. Implementei endpoints de diagnóstico de saúde (/health) e de métricas no padrão Prometheus (/metrics). As métricas incluem o volume de requisições HTTP por rota e código de status, o tempo de resposta e o consumo de CPU. O endpoint /health do catalogo_service verifica o banco de dados; o do log_service verifica a conexão com o Redis. Quando o Redis está indisponível, o endpoint do log_service responde com HTTP 503 e estado unhealthy; o health check do Docker interpreta essa resposta e marca o container como unhealthy. Esse mecanismo sinaliza o problema, mas não reinicia automaticamente o container. As Figuras 31 e 32 mostram, respectivamente, as métricas e o estado da stack antes da falha; a Figura 33 mostra o log_service como unhealthy com o Redis parado; e a Figura 34 mostra que o catálogo continuou respondendo com HTTP 200. A entrega foi finalizada em 12/09/2026 às 19:35 (commit a57e484).

```
commit a57e48436cf6e189e703b09d7efcf54e4bc07d43
Author: Gabriel Castão Graciano <gbraciano@hotmail.com>
Date: Sat Sep 12 19:35:18 2026 -0300

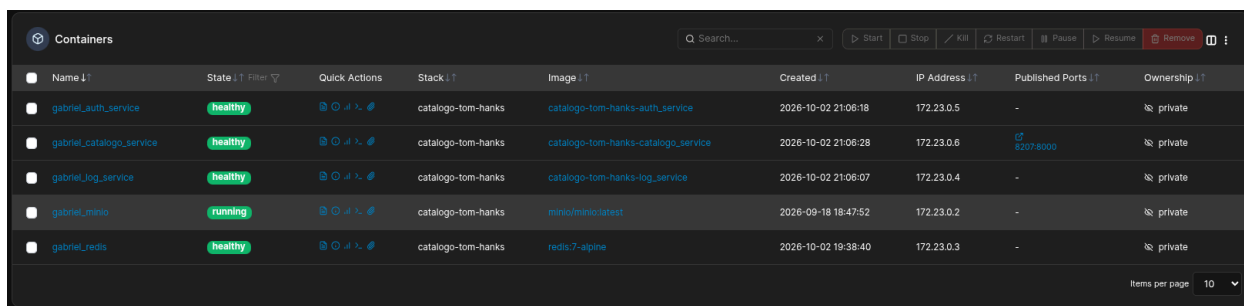
docs: adiciona evidencias e documentacao completa de observabilidade

diff --git a/README.md b/README.md
index 1108967..13f2df4 100644
--- a/README.md
+++ b/README.md
@@ -14,6 +14,36 @@
---
```

Figura 30: Atividade E3 — evidência da entrega (commit a57e484 com data e hora visíveis)

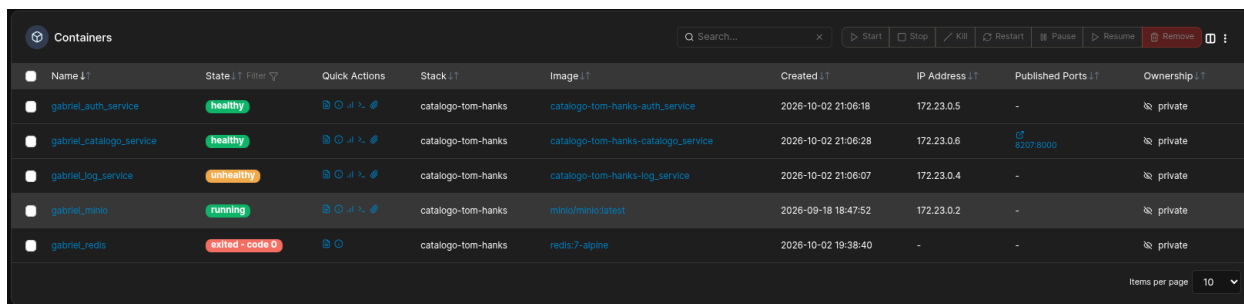
```
== /health ==
{"status":"healthy","service":"catalogo_service","dependencies":{"database":"up"}}
== requisicoes por rota e status ==
http_requests_total{handler="/health",method="GET",status="2xx"} 11727.0
http_requests_total{handler="/metrics",method="GET",status="2xx"} 3.0
http_requests_total{handler="/",method="GET",status="2xx"} 9.0
http_requests_total{handler="/api/perfil/me",method="GET",status="4xx"} 7.0
http_requests_total{handler="/api/comentarios/{tmdb_movie_id}",method="GET",status="4xx"} 6.0
http_requests_total{handler="/api/auth/login",method="POST",status="4xx"} 2.0
http_requests_total{handler="/api/auth/login",method="POST",status="2xx"} 7.0
http_requests_total{handler="/api/filmes",method="GET",status="2xx"} 5.0
http_requests_total{handler="/api/favoritos",method="GET",status="2xx"} 4.0
http_requests_total{handler="/api/favoritos",method="POST",status="2xx"} 6.0
http_requests_total{handler="/api/favoritos/{favorito_id}",method="DELETE",status="2xx"} 1.0
http_requests_total{handler="/api/perfil/me",method="GET",status="2xx"} 10.0
http_requests_total{handler="/api/perfil/{target_usuario_id}/foto",method="POST",status="2xx"} 1.0
http_requests_total{handler="/api/perfil/foto/{object_name}",method="GET",status="2xx"} 5.0
http_requests_total{handler="/api/auth/login",method="POST",status="4xx"} 3.0
http_requests_total{handler="/api/auth/logout",method="POST",status="2xx"} 2.0
http_requests_total{handler="/api/perfil/{target_usuario_id}",method="PUT",status="2xx"} 2.0
http_requests_total{handler="/api/perfil/{target_usuario_id}",method="PUT",status="4xx"} 1.0
== tempo de resposta (contagem e soma, em segundos) ==
http_request_duration_seconds_count{handler="/health",method="GET"} 11727.0
http_request_duration_seconds_sum{handler="/health",method="GET"} 36.93882616948599
http_request_duration_seconds_count{handler="/metrics",method="GET"} 4.0
http_request_duration_seconds_sum{handler="/metrics",method="GET"} 0.03456121002091095
http_request_duration_seconds_count{handler="/",method="GET"} 9.0
http_request_duration_seconds_sum{handler="/",method="GET"} 0.01349711102375295
http_request_duration_seconds_count{handler="/api/perfil/me",method="GET"} 17.0
http_request_duration_seconds_sum{handler="/api/perfil/me",method="GET"} 0.08294436200230848
http_request_duration_seconds_count{handler="/api/comentarios/{tmdb_movie_id}",method="GET"} 6.0
http_request_duration_seconds_sum{handler="/api/comentarios/{tmdb_movie_id}",method="GET"} 0.011389566819917838
http_request_duration_seconds_count{handler="/api/auth/login",method="POST"} 2.0
http_request_duration_seconds_sum{handler="/api/auth/login",method="POST"} 0.0041249659989262
== CPU ==
process_cpu_seconds_total 174.73
```

Figura 31: Atividade E3 — resultado (/health do catálogo com status healthy e banco up; /metrics no padrão Prometheus com requisições por rota e código de status, tempo de resposta (contagem e soma, em segundos) e consumo de CPU; contadores acumulados, com a maior parte em /health, consultado pelo healthcheck do Docker)



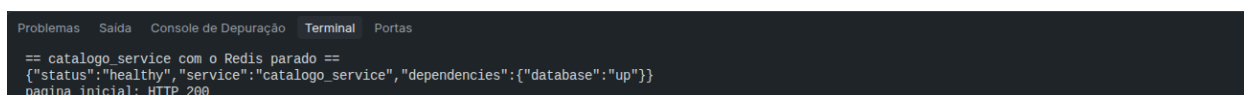
Name	State	Quick Actions	Stack	Image	Created	IP Address	Published Ports	Ownership
gabriel_auth_service	healthy	[Icons]	catalogo-tom-hanks	catalogo-tom-hanks-auth_service	2026-10-02 21:06:18	172.23.0.5	-	private
gabriel_catalogo_service	healthy	[Icons]	catalogo-tom-hanks	catalogo-tom-hanks-catalogo_service	2026-10-02 21:06:28	172.23.0.6	8207:8000	private
gabriel_log_service	healthy	[Icons]	catalogo-tom-hanks	catalogo-tom-hanks-log_service	2026-10-02 21:06:07	172.23.0.4	-	private
gabriel_minio	running	[Icons]	catalogo-tom-hanks	minio/minio:latest	2026-09-18 18:47:52	172.23.0.2	-	private
gabriel_redis	healthy	[Icons]	catalogo-tom-hanks	redis:7-alpine	2026-10-02 19:38:40	172.23.0.3	-	private

Figura 32: Atividade E3 — resultado (painel do Portainer com os serviços da stack em estado healthy; o MinIO não define healthcheck e aparece como running)



Name	State	Quick Actions	Stack	Image	Created	IP Address	Published Ports	Ownership
gabriel_auth_service	healthy	[Icons]	catalogo-tom-hanks	catalogo-tom-hanks-auth_service	2026-10-02 21:06:18	172.23.0.5	-	private
gabriel_catalogo_service	healthy	[Icons]	catalogo-tom-hanks	catalogo-tom-hanks-catalogo_service	2026-10-02 21:06:28	172.23.0.6	8207:8000	private
gabriel_log_service	unhealthy	[Icons]	catalogo-tom-hanks	catalogo-tom-hanks-log_service	2026-10-02 21:06:07	172.23.0.4	-	private
gabriel_minio	running	[Icons]	catalogo-tom-hanks	minio/minio:latest	2026-09-18 18:47:52	172.23.0.2	-	private
gabriel_redis	exited - code 0	[Icons]	catalogo-tom-hanks	redis:7-alpine	2026-10-02 19:38:40	-	-	private

Figura 33: Atividade E3 — resultado (com o Redis parado, o log_service passa a unhealthy, pois seu /health responde 503 quando a dependência falha)



```

Problemas Saída Console de Depuração Terminal Portas
== catalogo_service com o Redis parado ==
{"status":"healthy","service":"catalogo_service","dependencies":{"database":"up"}}
pagina inicial: HTTP 200

```

Figura 34: Atividade E3 — resultado (capturada com o Redis parado: o catalogo_service segue saudável e a página inicial responde 200)

Dificuldades e como foram resolvidas. O ponto mais sensível foi garantir que o diagnóstico reportasse falhas de dependência sem travar o serviço. Resolvi usando um timeout curto (2 segundos) na conexão com o Redis e capturando a exceção dentro do /health, que passa a responder 503 com o detalhe do erro em vez de lançar uma falha. Como o healthcheck do Docker apenas sinaliza o estado, o container não é reiniciado automaticamente. Durante o teste, o catálogo, cujo /health não depende do Redis, continuou respondendo normalmente.

6 Considerações finais

O percurso deste bimestre permitiu aprofundar a aplicação de conceitos de computação em nuvem e arquiteturas distribuídas. A evolução do catálogo de filmes permitiu aplicar, na prática, isolamento de redes internas com Docker, segregação de responsabilidades entre microsserviços, controle de acessos por papel (RBAC), auditoria centralizada com Redis e desacoplamento de arquivos binários com MinIO. A Atividade 1, de nivelamento em Flask, não foi realizada; portanto, o relatório não a apresenta como etapa concluída de uma migração de monólito. As entregas complementares — pipeline de integração contínua com testes e publicação automatizada de imagens no GHCR (com deploy da stack manual pelo Portainer), documentação interativa com OpenAPI e observabilidade com health checks e métricas no padrão Prometheus — reforçaram competências essenciais para a engenharia de software atual.

Quatro aprendizados práticos merecem destaque. Primeiro, o healthcheck do Docker sinaliza o container como unhealthy, mas não o reinicia automaticamente. Segundo, a persistência não é automática: sem volume, os bancos SQLite se perderam quando a stack foi recriada. Terceiro, a falta de uma variável de

ambiente pode causar erros pouco evidentes, como o envio de e-mail que falhava porque as credenciais do Mailtrap não chegavam ao container, e o log do container foi decisivo no diagnóstico. Quarto, uma evidência só vale pelo que demonstra, e por isso o relatório indica, em cada atividade, o que as capturas não comprovam.

A maior dificuldade técnica foi coordenar a segurança entre múltiplos containers em rede privada: validar tokens JWT de forma distribuída, encaminhar arquivos do MinIO sem expor a API diretamente à internet e gerenciar variáveis de ambiente e segredos. Ao refazer as evidências, identifiquei dois pontos fracos que pretendo corrigir. O primeiro é que parte das credenciais ainda está definida diretamente no `docker-compose.yml`; a melhoria é movê-las para variáveis da stack no Portainer e trocar os valores padrão, em especial a chave do JWT. O segundo é que os bancos SQLite ficavam dentro do container, sem volume, e os usuários de teste foram perdidos quando a stack foi recriada; a correção é persistir os bancos em volumes do Docker. Também teria sido útil estruturar testes de integração automatizados desde o início. Para as próximas etapas, pretendo aprofundar orquestração avançada, gateways de serviços, balanceamento de carga e estratégias de entrega em ambientes de nuvem de larga escala.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Documento assinado digitalmente
 **GABRIEL CASTAO GRACIANO**
Data: 07/10/2026 18:16:36-0300
Verifique em <https://validar.iti.gov.br>

Gabriel Castão Graciano

Pompeia, 07/10/2026